

```
1  # -*- coding: utf-8 -*-
2  """Assignment 2.ipynb
3
4  Automatically generated by Colab.
5
6  Original file is located at
7      https://colab.research.google.com/drive/1tZT6rBtq-wGEUwcf1F2NjZqvaM5Ymx-y
8
9  # Theoretical Questions
10
11  1. What are the key differences between SQL and NoSQL databases.
12
13  -SQL Databases (Relational)
14
15  Structure: Table-based (rows & columns)
16
17  Schema: Fixed (predefined structure)
18
19  Language: Use Structured Query Language (SQL)
20
21  Examples: MySQL, PostgreSQL
22
23  Best for: Structured data, complex queries, transactions (e.g., banking systems)
24
25  Scalability: Vertically scalable (increase server power)
26
27  NoSQL Databases (Non-relational)
28
29  Structure: Flexible (document, key-value, graph, column)
30
31  Schema: Dynamic (no fixed structure)
32
33  Language: No standard query language
34
35  Examples: MongoDB, Cassandra
36
37  Best for: Large-scale, unstructured or semi-structured data (e.g., big data,
38  real-time apps)
39  Scalability: Horizontally scalable (add more servers)
40
41  2.What makes MongoDB a good choice for modern applications?
42
43  MongoDB is popular for modern applications because it's built for flexibility, speed,
44  and scalability:
45
46  Flexible schema: Stores data in JSON-like documents, so structure can change easily
47  without redesigning tables
48
49  High scalability: Supports horizontal scaling (sharding), ideal for growing apps and
50  large data
51
52  Fast performance: Optimized for read/write operations, good for real-time apps
53
54  Handles unstructured data: Works well with varied data types (e.g., user data, logs,
55  content)
56
57  Developer-friendly: Uses simple, intuitive document format (similar to JavaScript)
58
59  3. Explain the concept of collections in MongoDB?
60
61  In MongoDB, a collection is a group of related documents.
62
63  It is similar to a table in SQL, but more flexible
64
65  A collection stores documents (records) in JSON-like format (BSON)
66  No fixed schema – documents in the same collection can have different fields
67  Collections are used to organize data logically (e.g., a "users" collection for user
68  data)
69
70  4.How does MongoDB ensure high availability using replication?
71
72  MongoDB ensures high availability through replication using a setup called a replica
```

```
67 set:
68 Replica set: A group of servers that store the same data
69
70 Primary node: Handles all read/write operations
71
72 Secondary nodes: Continuously copy (replicate) data from the primary
73
74 Automatic failover: If the primary fails, one secondary is automatically elected as
the new primary
75
76 Data redundancy: Multiple copies of data prevent data loss
77
78 5.What are the main benefits of MongoDB Atlas?
79
80 MongoDB Atlas offers several key benefits:
81
82 Fully managed: No need to handle setup, backups, or maintenance
83
84 Easy scalability: Scale up/down automatically based on demand
85
86 High availability: Built-in replication and automatic failover
87
88 Strong security: Includes encryption, access control, and monitoring
89
90 Multi-cloud support: Runs on AWS, Google Cloud, and Microsoft Azure
91
92 6. What is the role of indexes in MongoDB, and how do they improve performance?
93
94 In MongoDB, indexes are special data structures that help the database find data
quickly without scanning every document.
95
96 Role of Indexes
97
98 -Store a small portion of data (like specific fields) in a sorted order
99 Act like a shortcut to locate documents efficiently
100
101 How They Improve Performance
102
103 -Faster queries: Quickly find matching documents instead of checking all records
104
105 -Efficient sorting: Help in sorting results without extra processing
106
107 -Better filtering: Speed up searches using conditions (e.g., find() queries)
108
109 7. Describe the stages of the MongoDB aggregation pipeline?
110
111 In MongoDB, the aggregation pipeline processes data in stages, where each stage
transforms the documents and passes the result to the next stage.
112
113 -->Common Stages
114
115 $match
116
117 Filters documents (like WHERE in SQL)
118
119 $group
120
121 Groups data and performs calculations (e.g., sum, count, average)
122
123 $project
124
125 Selects specific fields or reshapes documents
126
127 $sort
128
129 Orders the documents (ascending/descending)
130
131 $limit
132
133 Limits the number of results
134
```

135 \$skip
136
137 Skips a number of documents (useful for pagination)
138
139 8. What is sharding in MongoDB? How does it differ from replication?
140
141 In MongoDB:
142
143 -Sharding
144
145 Definition: Splits large data into smaller parts and distributes them across multiple servers
146
147 Purpose: Handles big data and high traffic
148
149 Benefit: Improves performance by spreading load (horizontal scaling)
150 -Replication
151
152 Definition: Copies the same data across multiple servers (replica set)
153
154 Purpose: Ensures data backup and high availability
155
156 Benefit: Prevents data loss and supports failover
157
158 -Key Difference:
159
160 Sharding = distributes data (for scalability)
161
162 Replication = duplicates data (for reliability)
163
164 9. What is PyMongo, and why is it used?
165
166 PyMongo is the official Python library used to interact with MongoDB.
167
168 -it is used:
169 Connects Python to MongoDB
170 Performs operations like insert, update, delete, and query
171 Supports database management (collections, indexes, etc.)
172 Easy integration with Python applications.
173
174 10. What are the ACID properties in the context of MongoDB transactions?
175
176 In MongoDB, ACID properties ensure reliable and consistent transactions:
177
178 -Atomicity
179 All operations in a transaction succeed or none do (no partial updates).
180
181 -Consistency
182 Data remains valid and follows defined rules before and after the transaction.
183
184
185 -Isolation
186 Transactions run independently without interfering with each other.
187
188 -Durability
189 Once committed, data is permanently saved—even if a system failure occurs.
190
191 11. What is the purpose of MongoDB's explain() function?
192
193 In MongoDB, the explain() function is used to analyze how a query is executed.
194
195 -Purpose:
196
197 -Shows the query execution plan
198
199 Indicates whether indexes are used or not
200
201 Provides details like execution time, scanned documents, and stages
202
203 -it's useful because:
204
205 Helps optimize queries

206
207 Identifies performance issues (e.g., full collection scans)
208
209 12. How does MongoDB handle schema validation.
210
211 In MongoDB, schema validation is used to control the structure of documents in a
collection.
212
213 -How it works:
214
215 - Uses validation rules (usually JSON Schema) when creating or modifying a collection.
216 - Ensures documents follow required fields, data types, and formats
217 - Applies rules during insert and update operations
218
219 -Validation modes:
220 - Strict (error): Rejects invalid documents
221 - Moderate (warn): Allows but logs warnings
222
223 13. What is the difference between a primary and a secondary node in a replica set?
224
225 In MongoDB, a replica set has different node roles:
226
227 - Primary Node:
228 Handles all write operations (insert, update, delete)
229 Also serves read requests (by default)
230 Acts as the main source of data
231 - Secondary Node:
232 Maintains a copy of data by replicating from the primary
233 Typically used for read operations (if enabled)
234 Can become primary during failover
235
236 14. What security mechanisms does MongoDB provide for data protection?
237
238 In MongoDB, several security mechanisms protect data:
239
240 - Authentication
241 Verifies user identity (username/password, SCRAM, LDAP, etc.).
242 - Authorization
243 Role-Based Access Control (RBAC) limits what users can do (read/write/admin).
244 - Encryption
245 Encryption at rest: Secures stored data.
246 Encryption in transit (TLS/SSL): Protects data while moving over networks.
247 - Auditing
248 Tracks and logs user activities for monitoring and compliance.
249 - Network Security
250 IP whitelisting, firewalls, and VPC support to restrict access.
251
252 15. Explain the concept of embedded documents and when they should be used?
253
254 In MongoDB, embedded documents (also called nesting) mean storing related data inside
a single document rather than in separate collections.
255
256 -Concept
257
258 A document can contain another document or array of documents.
259 Example: A "user" document containing an address object or order list inside it
260
261 -When to Use
262
263 - When data is closely related and accessed together
264
265 - For one-to-one or one-to-few relationships
266
267 - To improve read performance (fewer queries needed)
268
269 16. What is the purpose of MongoDB's \$lookup stage in aggregation?
270
271 In MongoDB, the \$lookup stage is used to join data from another collection during
aggregation.
272
273 -Purpose:
274 - Combines documents from two collections based on a common field

275 - Works similar to a JOIN in SQL
276 - Adds matched data as a new field (array) in the result
277
278 -When to use:
279 - When related data is stored in separate collections
280 - For reference-based relationships (instead of embedded data)
281
282 17. What are some common use cases for MongoDB?
283
284 MongoDB is widely used in modern applications due to its flexibility and scalability.
Common use cases include:
285
286 - Web & Mobile Applications
287
288 -Stores user data, sessions, and app content
289
290 -Handles dynamic and rapidly changing data
291
292 - E-commerce Platforms
293
294 -Manages product catalogs, orders, and customer data
295
296 -Supports flexible product attributes (size, color, etc.)
297
298 - Big Data & Real-Time Analytics
299
300 -Handles large volumes of unstructured or semi-structured data
301
302 -Useful for dashboards and real-time insights
303
304 - Content Management Systems (CMS)
305
306 -Stores articles, media, and user-generated content
307
308 -Flexible schema fits changing content needs
309
310 - Internet of Things (IoT)
311
312 -Stores sensor and device data streams
313
314 -Handles high-speed data generation
315
316 - Logging & Monitoring Systems
317
318 -Collects and analyzes logs, events, and metrics
319
320 18. What are the advantages of using MongoDB for horizontal scaling?
321
322 MongoDB is well-suited for horizontal scaling (adding more servers) mainly through
sharding.
323
324 -Advantages
325 - Handles large data easily
326 - Distributes data across multiple servers, avoiding single-server limits
327 - Improved performance
328 - Workload is shared, so queries and writes are faster
329 - High availability
330 - Combined with replication, ensures system stays up even if one node fails
331 - Cost-effective scaling
332 - Add cheaper servers instead of upgrading one expensive machine
333 - Flexible growth
334 - Scale out as your application grows without major redesign
335
336 19. How do MongoDB transactions differ from SQL transactions?
337
338 -MongoDB Transactions
339 - Designed for flexible, document-based data
340 - Supports multi-document transactions (in replica sets/sharded clusters)
341 - Often not required due to embedded documents (data stored together)
342 - Slightly higher overhead when used
343 - SQL Transactions
344 - Built for structured, relational data

- 345 - Transactions are core and widely used
- 346 - Strong ACID compliance by default
- 347 - Ideal for complex multi-table operations (e.g., banking systems)

348 -Key Difference:

349 MongoDB: Flexible, transactions optional

351 SQL: Structured, transactions essential

352

353 20. What are the main differences between capped collections and regular collections?

354

355 In MongoDB, capped collections and regular collections differ mainly in size and behavior:

356

357 -Capped Collections

- 358 - Fixed size (predefined limit)
- 359 - Store data in insertion order (FIFO)
- 360 - When full, old documents are automatically overwritten
- 361 - Faster for high-throughput inserts
- 362 - No document deletion allowed (automatic overwrite only)

363

364 -Regular Collections

- 365 - No size limit (can grow dynamically)
- 366 - Support insert, update, delete operations freely
- 367 - No automatic data removal
- 368 - More flexible for general use

369

370 21. What is the purpose of the \$match stage in MongoDB's aggregation pipeline?

371

372 In MongoDB, the \$match stage is used to filter documents in the aggregation pipeline.

373

374 - Purpose:

375

376 Selects only the documents that meet specific conditions

377

378 Works like a WHERE clause in SQL

379

380 Reduces the amount of data passed to later stages

381

382 - Benefit:

383

384 Improves performance by processing only relevant data

385

386 22. How can you secure access to a MongoDB database?

387

388 - Authentication

389

390 Enable login using username & password (SCRAM, LDAP, etc.)

391

392 - Authorization

393

394 Use Role-Based Access Control (RBAC) to limit user permissions (read/write/admin)

395

396 - Encryption

397

398 In transit: Use TLS/SSL to protect data over the network

399

400 At rest: Encrypt stored data on disk

401

402 - Network Security

403

404 Restrict access using IP whitelisting, firewalls, or VPCs

405

406 Avoid exposing the database directly to the public internet

407

408 - Auditing & Monitoring

409

410 Track user activities and detect suspicious behavior

411

412 23.What is MongoDB's WiredTiger storage engine, and why is it important?

413

414 In MongoDB, WiredTiger is the default storage engine responsible for how data is stored, managed, and accessed on disk.

```
415
416 - it's important:
417
418 1. High performance: Uses caching and compression for faster read/write operations
419
420 2. Efficient storage: Compresses data to save disk space
421 3. Concurrency control: Allows multiple operations at the same time (document-level
locking)
422 4. Crash recovery: Uses journaling to prevent data loss
423 """
```